

Computer Network

Lab 4: Socket programming

Contents of the document:

1. Understanding Socket Programming
2. Client-Server Communication Model
3. Data Transmission Protocols
4. Understanding Sockets
5. Socket Types
6. Socket Programming in C
7. Socket APIs (All routes for TCP & UDP)
8. Byte Ordering
9. Server Side program
10. Client Side Program
11. Compile and execute socket programs

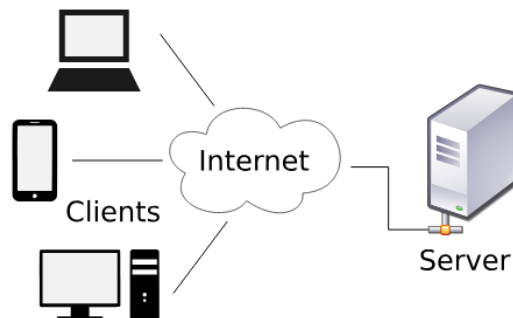
I. Understanding socket programming

- A socket is a software object that allows a program to send and receive data over a network. Socket acts as an endpoint for communication, it is uniquely programmed by IP address (identifies which machine) and port number (identifies which application on that machine).
- Socket programming lets two programs communicate through sockets which are identified by IP address (32 bit) and port number (16 bit) which gives a 48-bit socket address.
- In socket programming, the OS already implements networking protocols such as TCP and UDP. The programmer uses socket APIs provided by OS to access these networking services.
- It follows the client-server model, where the server waits for connection requests as input and the client initiates the communication by connecting to the server. Depending on the type of socket used, the communication can be reliable and connection oriented (TCP) or fast and connectionless (UDP).

II. Client-Server communication model

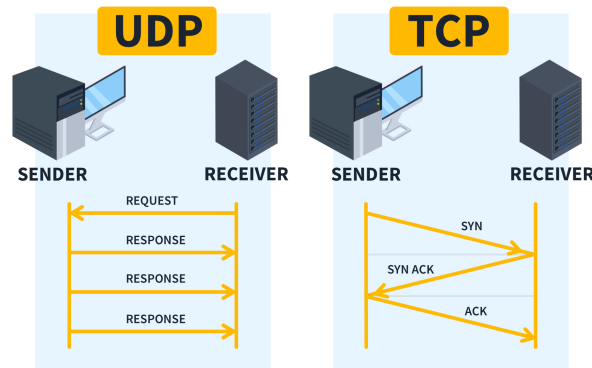
- The model is a network architecture in which communication takes place between two types of programs - a client and a server
- The server is a program that provides services and resources and a client is a program that requests those services. The server runs continuously and waits for clients requests, while the client is responsible for initiating the communication in need of a service.
- Working of the client server model is as follows:
 - The server creates a socket

- The server binds the socket to a specific port number and waits for incoming requests.
 - The client creates its own socket and sends a request to the server using the server's IP address and port number.
 - When communication is established, data is exchanged (as bytes) between the client and the server.
 - After the communication is completed, the connection is closed.
- The role of the server is to listen for incoming client connections and accept connection requests. It processes client requests and sends responses back to the client.
- The role of the client is to initiate the communication. It connects to the server's IP address and port and sends requests and receives responses from the server.
- The client server model enables a centralized control of data and services. It can support multiple clients communicating with a single server. It forms the basis of most network applications such as web browsing, email and file transfer.



III. Data Transmission Protocols

- The data flows through the sockets as bytes. Text, image, numbers, files or anything are converted to bytes and transferred through sockets. These protocols ensure communication between network systems is carried out correctly and efficiently.
- When data flows, some rules are needed, also called protocols. The important protocols are TCP and UDP.
- Transmission control protocol (TCP) is a reliable, ordered and connection oriented protocol. It is slow but safe for applications like email, HTTP, file transfer etc. TCP is used for error detection and correction. In socket programming, TCP uses stream sockets (SOCK_STREAM) and requires functions such as listen() and accept() on the server side.
- User datagram protocol (UDP) is a fast, lightweight, connectionless protocol. It provides no guarantee in the transmission of all data packets in order sequence used in applications like video calls, games, live streaming etc. UDP uses datagram sockets (SOCK_DGRAM) and doesn't require listen() and accept() functions.



IV. Understanding Sockets

- A socket is an endpoint for communication between two programs over a network. It provides a way for applications to send and receive data using standard networking protocols such as TCP and UDP.
- A socket is an interface between the application layer and transport layer. The OS uses sockets to manage network communication efficiently.
- The socket allows a program to establish connection with another program. It sends and receives data.
- The socket is uniquely identified by IP address which identifies the machine and port number which identifies the application or service on that machine. This ensures data reaches the correct destination program on the network.
- The server socket waits for incoming client requests. The client sockets initiates the communication. Once the communication is established, sockets enable a two-way communication for data transfer between client and server.
- Sockets hide the complexity of the underlying network and provide a simple programming interface for network communication.

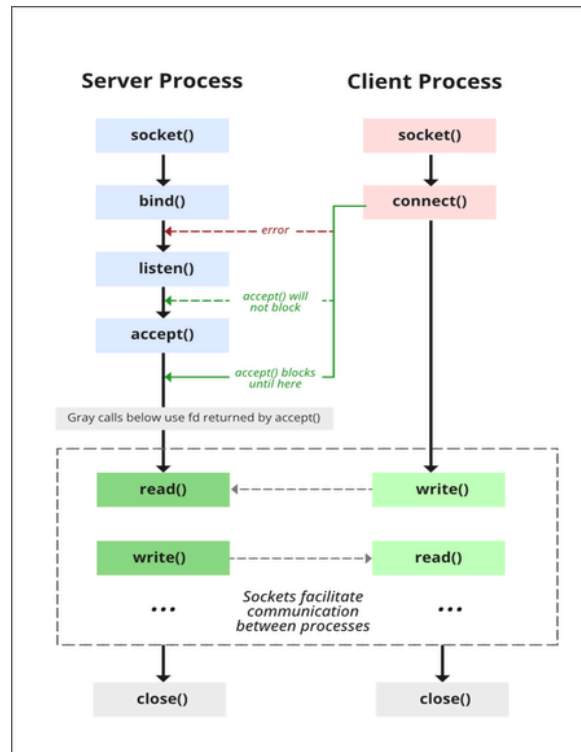
V. Socket Types

- The sockets are classified based on how data is transmitted between communicating programs. The most commonly used socket types are Stream sockets and Datagram sockets.
- Stream sockets:
 - These provide reliable, two way, connection oriented communication between client and server. They are used with TCP.
 - Stream sockets are used in applications where data accuracy and order are important such as web applications, file transfer, email communication etc.
 - Stream sockets are created using `SOCK_STREAM`
- Datagram sockets:

- These provide connectionless communication and are mainly used in UDP. It provides lower overhead but no guarantee in delivery or order.
- Datagram sockets are commonly used in applications where speed is more important than reliability such as video streaming, online gaming, voice communication etc.
- Datagram sockets are created using SOCK_DGRAM

VI. Socket programming in C

- Socket programming in C allows applications to communicate over a network using system defined socket APIs. C language provides low-level access to these APIs making it suitable for building efficient and high performance network applications.
- In C, socket programming is done using a set of functions provided by the OS. These functions allow programmers to create sockets, establish connections, send and receive data and close connections.
- Socket programming steps:
 - Server side:
 - Create a socket - using socket() function
 - Bind the socket - the socket is bound to a specific IP address and port number using bind() function
 - Listen for connections - the server waits for client connection requests using listen() function
 - Accept a connection - when a client requests a connection, the server accepts it using the accept() function
 - Send and receive data - data is exchanged using send() and recv() functions
 - Close the socket connection - after communication is complete, the socket is closed using close()
 - Client side:
 - Create a socket - the client creates a socket using the socket() function
 - Connect to the server - the client connects to server using server's IP address and port number with connect() function
 - Send and receive data - data is exchanged using send() and recv()
 - Close the socket - the connection is closed using the close() function
- Using C for socket programming provides fine control over system resources. It is efficient and fast execution. It enables direct access to OS networking APIs. It is widely used for system-level and network applications.



VII. Socket APIs

- Socket APIs are a set of system defined functions provided by the OS that allow programs to perform network communication. These APIs hide the internal complexity of networking protocols and provide a standard interface to create, manage, and terminate network connections.
- TCP socket API route
 - TCP is a connection oriented protocol so it requires a well-defined sequence of API calls
 - TCP server API flow: socket() -> bind() -> listen() -> accept() -> send()/recv() -> close
 - TCP client API flow: socket() -> connect() -> send()/recv() -> close()
- UDP socket API flow
 - UDP is connectionless protocol so fewer socket API calls are required
 - UDP server API flow: socket() -> bind() -> recvfrom()/sendto() -> close()
 - UDP client API flow: socket() -> sendto()/recvfrom() -> close()

VIII. Byte ordering

- Byte ordering, also known as endianness, refers to the order in which multi-byte data is stored and transmitted in computer systems. Different systems may store data in different byte orders, which can cause inconsistencies during network communication.

- There are two types of byte ordering:
 - Little Endian: Least significant byte is stored first. It is commonly used by Intel-based systems.
 - Big Endian: Most significant byte is stored first. It is used by network protocols
- Network Byte Order: In networking, data is always transmitted in Big Endian format, which is known as Network Byte Order. The byte order used by the local machine is called Host Byte Order. Since the host byte order may differ from the network byte order, conversion is required before transmitting data over a network.
- Byte Order Conversion Functions: C provides built-in functions to convert between host and network byte order. These functions are commonly used when setting port numbers and IP addresses in socket structures.
 - htons() - Host to Network Short (16-bit)
 - htonl() - Host to Network Long (32-bit)
 - ntohs() - Network to Host Short
 - ntohl() - Network to Host Long
- Byte ordering ensures correct interpretation of numerical data. It prevents communication errors between different systems. It is essential for portability in network applications

IX. Server Side Program (TCP-C)

- TCP server program:


```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

int main() {
    int server_fd, new_socket;
    struct sockaddr_in server_addr, client_addr;
    socklen_t addr_len = sizeof(client_addr);
    char buffer[1024] = {0};
    char message[] = "Hello from Server";

    // 1. Create socket
    server_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (server_fd < 0) {
        perror("Socket creation failed");
        exit(1);
    }
}
```

```

// 2. Bind socket to IP and port
server_addr.sin_family = AF_INET;
server_addr.sin_addr.s_addr = INADDR_ANY;
server_addr.sin_port = htons(8080);

bind(server_fd, (struct sockaddr*)&server_addr, sizeof(server_addr));

// 3. Listen for connections
listen(server_fd, 5);

printf("Server listening on port 8080...\n");

// 4. Accept client connection
new_socket = accept(server_fd, (struct sockaddr*)&client_addr, &addr_len);

// 5. Receive data from client
recv(new_socket, buffer, sizeof(buffer), 0);
printf("Client says: %s\n", buffer);

// 6. Send response to client
send(new_socket, message, strlen(message), 0);

// 7. Close sockets
close(new_socket);
close(server_fd);

return 0;
}

```

X. Client side program (TCP - C)

- TCP client program


```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

int main() {
    int sock;
    struct sockaddr_in server_addr;
    char buffer[1024] = {0};
    char message[] = "Hello from Client";

```

```

// 1. Create socket
sock = socket(AF_INET, SOCK_STREAM, 0);
if (sock < 0) {
    perror("Socket creation failed");
    exit(1);
}

// 2. Server address
server_addr.sin_family = AF_INET;
server_addr.sin_port = htons(8080);
server_addr.sin_addr.s_addr = inet_addr("127.0.0.1");

// 3. Connect to server
connect(sock, (struct sockaddr*)&server_addr, sizeof(server_addr));

// 4. Send message to server
send(sock, message, strlen(message), 0);

// 5. Receive reply from server
recv(sock, buffer, sizeof(buffer), 0);
printf("Server says: %s\n", buffer);

// 6. Close socket
close(sock);

return 0;
}

```

XI. Compilation and execution of socket programs

- To compile:
 - gcc server.c -o server
 - gcc client.c -o client
- To run:
 - ./server (in terminal 1)
 - ./client (in terminal 2)
- Output:

```
▼ TERMINAL
● bhaavya@DESKTOP-H5EGEGB:/mnt/c/Users/Bhaavya/Desktop/CN/socket-programming$ ./server
Server listening on port 8080...
Client says: Hello from Client
○ bhaavya@DESKTOP-H5EGEGB:/mnt/c/Users/Bhaavya/Desktop/CN/socket-programming$
```

```
▼ TERMINAL
● bhaavya@DESKTOP-H5EGEGB:/mnt/c/Users/Bhaavya/Desktop/CN/socket-programming$ ./client
Server says: Hello from Server
○ bhaavya@DESKTOP-H5EGEGB:/mnt/c/Users/Bhaavya/Desktop/CN/socket-programming$
```

Socket programs were executed using GCC in a WSL linux environment through VSCode

Submitted By: 25MCB1010, Patsamatla Bhaavya Ramita

Email: patsamatla.bhaavya@vitstudent.ac.in